

从 oh-my-cli 到 one-cli: Coding Agent 深度研究与 0→1 实践

基于 676 个提交、数百个 Issue 线索、核心运行时源码与测试体系，还原一个 Coding Agent 如何从工具循环演进为可审计平台，并将关键不变量落实为独立实现。

核心结论：Coding Agent 的成熟度不取决于工具数量，而取决于 partial stream、tool-call 配对、crash、cancel、resume、workspace 边界和自动化协议是否可恢复、可解释、可测试。

研究对象

qwen-code-dev-bot/oh-my-cli · main@38ed637

实践项目

beforeload/one-cli · main@6a8071d

报告日期

2026-08-07

报告范围

架构、Issue 演化、安全、工程质量、绿地实现

执行摘要

oh-my-cli 的真实定位并非一个“small code-agent CLI”，而是一套围绕安全控制面、持久会话、自动化协议和自治研发流程构建的实验性 Coding Agent 平台。它最值得借鉴的是边界语义；最需要警惕的是文档叙事、默认运行时和扩展整合之间的落差。

676

oh-my-cli main 历史提交

298±

本地证据可追踪的 Issue 实现

399

测试 TypeScript 文件

98.1%

Bot 提交占比

最值得保留

- Transcript integrity first：任何退出都不留下 orphan tool call。
- Provider 只在零输出前重试；partial output 必须持久化。
- Hard policy 在 approval 之前，便利模式不能绕过结构性禁止。
- Session 与 workspace 绑定；损坏状态拒绝静默恢复。
- stdout 是 versioned JSONL 协议，不是随意日志。

最关键的落差

- Folder trust 模块 fail-closed，但主 CLI enforcement 默认关闭。
- Shell 只是 cwd 固定，并没有进程内 OS sandbox。
- MCP/custom tools 可检查、可单次调用，却未进入主 agent loop。
- index.ts 与 tui-shell.ts 过度集中。
- npm 发布内容未收口，测试、源码与治理文件一并进入包。

0→1 落地结果

one-cli 已将关键不变量压缩成一个可运行的 TypeScript MVP：12 个核心源码模块、6 个内置工具、四档审批、append-only session、workspace-bound resume、versioned JSONL、24 个单元测试、6 个进程级集成测试；真实构建产物连接本地 OpenAI-compatible SSE 服务运行成功，退出码为 0。

目录

01	研究范围与证据方法	07	可迁移模式与反模式
02	仓库画像与产品判断	08	one-cli 绿地架构
03	Issue 驱动的九阶段演化	09	实现与验证证据
04	核心运行时与不变量	10	后续路线图
05	安全模型与风险登记	11	结论
06	工程质量与治理	A	Issue 与代码证据索引

01 研究范围与证据方法

研究对象为本地仓库 `/Users/daniel/workspace/oh-my-cli`，快照 `main@38ed637`；实践对象为 `github.com/beforeload/one-cli`。本报告将“代码事实”“历史事实”与“架构推理”分层处理。

证据类型	使用方式	置信度
源码与调用关系	读取 agent、provider、tools、workspace、session、approval、headless 等核心模块	高
测试与 CI	单元/集成/smoke 文件、workflow、package scripts、实际 typecheck/pack/audit	高
Git 历史	676 个 commit、320 个 merge commit、Issue 分支 slug、commit body	高
Issue 重建	通过 Issue #N 注释、分支名、merge 标题与父子 roadmap 线索还原	中高
Live Issue body/label/comment	研究时 GitHub CLI 凭据失效，未读取；不把推理标题冒充原文	未纳入

解释边界

“Issue 编号、实现文件、测试、合并时间”属于可验证事实；阶段名称、部分父 Issue 主题和未合并编号状态属于推理。所有推理均以本地证据为边界。

02 仓库画像与产品判断

195 src TypeScript 文件	247 unit 测试文件	148 integration 测试文件	100+ Commander 选项面
--------------------------	------------------	-------------------------	-----------------------

项目已经覆盖 TUI、Headless、Electron Desktop、本地 Web、Provider/MCP/tool/workflow contract、Session 运维、Goal/Mission、Subagent/Worktree 与自治研发治理。版本仍为 0.1.0，但能力面已经是平台级。

层	实际能力	判断
交互层	全屏 TUI、readline fallback、Electron Desktop、本地 delivery web	多个 surface 共用核心，但能力不完全对齐
Agent 核心	流式响应、7 个工具、重试、fallback、预算、取消、最多 30 轮	边界清楚，逻辑集中在 <code>agent.ts</code>

层	实际能力	判断
安全层	审批、command policy、hooks、path confinement、folder trust、read-only	防线丰富，但默认值存在关键接缝
状态层	JSONL、恢复、压缩、撤销/重做、归档、bundle、journal、evidence	最成熟、最有差异化的部分
扩展层	Provider/MCP/tool/workflow contract、发现、健康检查、一次性调用	治理优先，主 agent-native 整合滞后
自治开发层	Issue lease、Bot 执行、governance plane、质量门禁	审计性强，所有权高度集中

产品判断

适合作为安全型 Coding Agent 的研究底座与原型；在 folder trust 默认 enforcement、OS sandbox、扩展整合、跨平台 CI 和发布收口完成前，不应仅凭 README 的安全叙事用于高权限、无人值守生产任务。

03 Issue 驱动的九阶段演化

本地历史覆盖 2026-07-13 至 2026-08-06，约 24 天。项目采用“一条不变量 + 一个纵向切片 + 对应测试”的高频演进方式，而不是先完成大规模平台设计。

- #1–#50 · Foundation**
CI、doctor、headless、session、subagent/worktree。先建立可运行、可恢复、可验证的执行壳。
- #51–#100 · Safety + TUI**
Command policy、folder trust、compaction、conversation shell。能力出现后立即补安全门。
- #101–#200 · Reliability + Extensions**
Symlink 防逃逸、MCP contract、extension discovery、Desktop 基础。先 contract，再 invoke。
- #201–#280 · Parallel + Headless Ops**
Read-only、partial stream、fatal boundary、quota、delivery web。围绕无人值守补终止语义。
- #281–#350 · Workbench**
Concept contract、activity、mission lifecycle、review studio。用纯 projection 支撑多 surface。
- #351–#450 · Goal System**
Goal constraint、approval、budget、conflict、queue。把任务意图建模为状态，而非 prompt 文本。
- #451–#550 · Limits + Desktop + Config**
Run caps、workspace env、zoom、salvage、Ctrl+C。围绕实际使用收紧边界。
- #551–#630 · Session Operations**
Fork/search/archive/inspect/notes/pin/journal。Session 从日志演化为运营对象。
- #631–#708 · Automation Hardening**
Journal 聚合、strict exit、store doctor、bundle verify。形成机器可判定接口。

Issue 的真实作用：定义运行时协议

Issue	可观察不变量	为什么重要
#243	Provider 已产生文本后失败：保留 partial assistant，禁止重试	避免重复文本与上下文丢失

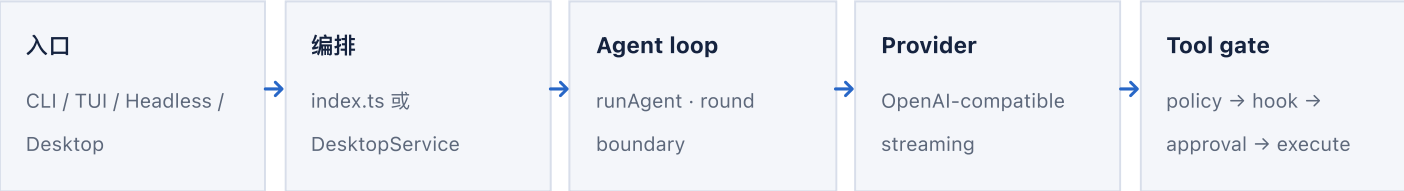
Issue	可观察不变量	为什么重要
#244	空 completion 只能有界重试	防止无限循环和 token 浪费
#247	Quota exhausted 与普通 429 分开分类	配额失败不应进入退避循环
#550	取消时每个 tool_call_id 仍有唯一 tool message	保证 transcript 可恢复和可再次提交
#552	Headless SIGINT 有终态 event, 退出码 130	让 CI/编排器准确识别取消
#554	Resume 与原 workspace 绑定	防止历史上下文跨仓库误操作
#590	Fallback model 只在零输出 transient failure 后触发一次	避免重复输出和路由震荡

Issue 设计的可迁移原则

不写“优化重试机制”这类抽象任务，而写“503 在首字节前最多重试 3 次；首字节后失败不得重试； retry 必须输出 typed event”这类可观察协议。

04 核心运行时与不变量

4.1 一个 loop，多个 surface



```
round boundary
→ cancel / budget / turns / wall-time / tool-call caps
→ provider stream
→ assistant text 或完整 tool-call batch
→ 工具顺序执行
→ 每个 call 写入唯一 tool result
→ 下一轮或稳定终止
```

4.2 Transcript integrity

情况	持久化语义
正常文本结束	一个 complete assistant message
Tool call	Assistant 保存完整 tool_calls, 之后逐个追加 tool message

情况	持久化语义
部分文本后 Provider 错误	Assistant 标记 interrupted, 不重试
零输出 transient error	Provider 层有界重试, 并输出 retry event
Tool batch 中途取消	未执行 call 追加 cancelled placeholder
预算/轮数/时间耗尽	只在稳定边界停止, 避免半个 batch

Provider

- SDK 自动重试关闭, 由 runtime 显式拥有 backoff。
- `producedOutput` 是是否可重试的硬边界。
- Retry、fallback、usage 都是结构化事件。
- 空响应重试与网络重试属于不同 reason class。

Session

- Message 增量 append, 尾部 torn line 可容忍。
- 中间坏行视为 corrupt, 隔离而非删除。
- Resume 检查 workspace identity。
- Salvage 复制可解析前缀, 保留原始证据。

4.3 Tool gate

```
unknown tool
→ read-only mode
→ folder trust
→ shell command policy
→ deny-only user hooks
→ approval mode
→ Zod parse / workspace resolve / execute
```

关键不是 gate 数量, 而是顺序: 结构性策略必须位于人工审批之前, `yolo` 只能跳过交互确认, 不能把已知危险形状变成安全操作。

4.4 Headless 是协议

- 每条记录包含 protocol/version/seq/ts; 消费者可检测缺失和乱序。
- stdout 只输出 NDJSON; 审批和人类诊断进入 stderr。
- Terminal record 的 exitCode 与进程退出码一致。
- Tool result 区分 succeeded / failed / cancelled。

05 安全模型与风险登记

最重要的事实

`folder-trust.ts` 本身按 fail-closed 设计，但主 CLI 只有在显式启用 `--enforce-folder-trust` 或环境开关时才把该决定传入 run。README 的“mutating tools fail closed by default”与默认执行路径并不完全一致。

做得好的地方

- **路径边界**：canonical path + symlink ancestor 检查。
 - **策略先于审批**：危险 shell shape 不可被 yolo 绕过。
 - **密钥最小化**：settings 保存 env 名，不保存 credential value。
- **失败可审计**：policy/hook/approval/provider/cancel 都有结构化状态。
 - **原子写**：同目录临时文件 + rename。
 - **Desktop hardening**：contextIsolation、sandbox、禁用 nodeIntegration。

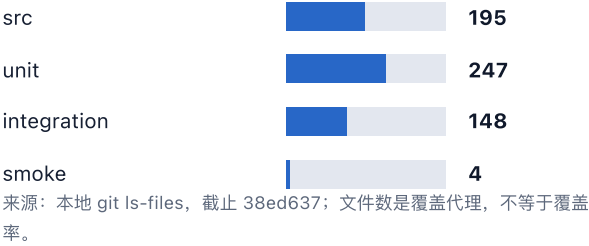
风险登记

级别	发现	影响	证据强度
高	Folder trust mutation enforcement 默认关闭	用户可能基于文档错误假设未信任仓库不可写	直接代码证据
高	Shell 无内建 OS sandbox	trusted + yolo 仍然具备主机级 shell 能力	直接代码证据
高	MCP/custom tools 不在 agent loop	配置扩展后模型仍无法自主调用	Import/call graph
中	主运行配置与 effective settings 分叉	审计快照和真实 run 消费配置可能不同	调用点检索
中	TUI history.slice(0,-1)	疑似造成交互上下文缺口	需补行为测试
中	Desktop 审批能力缺口	文件可自动编辑，shell 永远拒绝，surface 不对齐	固定 handler

Shell 边界必须诚实描述

把 `cwd` 固定到 workspace 不能限制绝对路径、`..`、子进程和网络。Command policy 只能拒绝已知危险形状；严格 confinement 需要 macOS sandbox、Linux namespace/container 或专用原生 helper。

代码与测试文件量



提交所有权



方面	证据	评价
类型与构建	TypeScript strict；typecheck 通过	稳健
测试	247 unit + 148 integration + 4 smoke	密度很高，缺 coverage threshold
CI	Build、typecheck、unit、integration、desktop、smoke、gitleaks、audit	门禁丰富，平台矩阵不足
依赖	3 个 runtime deps；Electron dev 链较重	核心供应链小
发布	无 tag、无自动 publish/provenance、手工 checklist	早期
代码组织	index.ts 5,678 行；tui-shell.ts 4,643 行；agent.ts 1,137 行	维护成本高
静态质量	无 lint、formatter、coverage job	测试承担过多压力
治理	320/676 为 merge commit；Bot 禁改 governance plane	可审计，所有权集中

发布收口问题

原项目没有 package.json.files allowlist；dry-run 显示 npm 包包含测试、源码、GitHub workflow 和自治治理文件。对 CLI 产品而言，这会放大包体、攻击面和发布审查成本。

07 可迁移模式与反模式

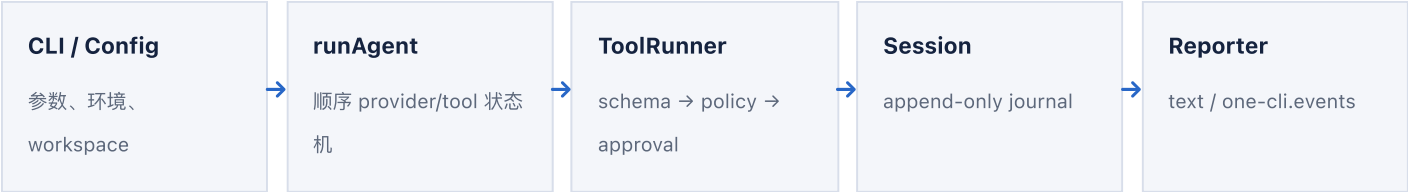
模式	保留原因	在新项目中的落实
Transcript integrity first	防止取消/崩溃产生无效 tool transcript	one-cli 对未执行 batch call 写 cancelled tool message
Retry only before output	防止 partial 文本重复	集成测试覆盖 503 零输出重试与 partial 后不重试
Hard policy before approval	用户确认不是安全证明	all 不能绕过 secret/.git/危险 shell 规则
Workspace authority	避免每个工具自行处理路径	文件访问统一经过 canonical + symlink gate
Append-only session	崩溃可审计，避免 checkpoint 覆盖证据	每条 record 带 schema/version/seq/ts
Machine contract	让 CI/编排器可靠消费	纯 stdout JSONL + 精确 0/1/2/130 exit code
Scripted provider seam	测试不依赖真实模型	本地 SSE fake server 驱动真实 dist CLI

不应复制的结构

1. 从第一天加入 TUI、Electron、Web、MCP、Goal、Journal 和 Subagent。
2. 把所有 option 与 dispatch 压进一个 5,000+ 行入口。
3. 把 UI raw mode、审批、状态机和 agent wiring 压进同一 TUI 文件。
4. 用自制 shell tokenizer 冒充完整安全边界。
5. 把 contract/discovery/invoke 描述成 agent-native extension。
6. 同时维护两套配置事实来源。
7. 把源码、测试、治理面全部发布到 npm。

08 one-cli 绿地架构

one-cli 不复制平台表面，只实现一条可运行、可恢复、可审计的纵向路径：prompt → provider → tool gate → side effect → durable session → final event。



产品边界

首版包含

- Node 22 / TypeScript ESM
- OpenAI-compatible streaming
- read/list/grep/write/edit/shell
- deny/ask/auto-edit/all
- Session、resume、JSONL、SIGINT

明确延后

- TUI / Desktop / Web
- MCP / plugins / skills
- Subagent / worktree
- Goal / journal / archive
- Compaction / profiles / telemetry

Tool runner 的固定顺序

```
lookup
→ Zod validation
→ hard policy
→ side-effect-free prepare
→ bounded preview
→ approval
→ stale-target hash recheck
→ execute
→ bounded result
```

oh-my-cli 与 one-cli 的取舍

主题	oh-my-cli	one-cli 决策
Surface	CLI/TUI/Desktop/Web	单一 CLI，先稳定 execution kernel
工具	7 个内置工具 + 平行扩展 invoke	6 个内置工具，暂不承诺扩展
Session	JSONL + checkpoint + 多 sidecar	严格 append-only，单文件记录
Folder trust	能力存在，默认 enforcement 接缝	所有 file tool 强制 workspace authority
Shell	cwd 固定 + policy	明确标注 host-capable，不宣称 sandbox
发布	无 files allowlist	只发布 dist、README、LICENSE、metadata
测试	大规模矩阵	少而关键，覆盖 runtime 边界

09 实现与验证证据

12 核心 src 模块	24 Unit cases · 全通过	6 Integration cases · 全通过	0 npm audit vulnerabilities
------------------------------------	---	---	---

核心实现

文件	职责	关键不变量
src/agent.ts	Provider/tool 顺序状态机	Tool-call 配对、partial 持久化、稳定终止
src/provider.ts	OpenAI-compatible streaming	零输出前重试，输出后不重试
src/tools.ts	Schema、gate、执行、shell runner	Policy → approval → stale recheck
src/workspace.ts	文件系统 authority	Traversal/symlink 防逃逸、atomic write
src/session.ts	Journal、lock、resume	Append-only、workspace binding、in-doubt guard
src/reporter.ts	Text / JSONL projection	Monotonic seq、stdout purity
src/approval.ts	ApprovalPort 与矩阵	非 TTY fail-closed
src/policy.ts	不可绕过的 hard deny	Secret、.git、危险 shell shape
src/cli.ts	Composition root	0/1/2/130 exit contract

进程级验证覆盖

- Final answer：真实 `dist/index.js` 输出完整终态。
- 503 retry：首字节前重试，输出 `provider.retry`。
- Partial stream：保留 partial assistant，且请求次数保持 1。
- Read tool round trip：模型请求工具，结果进入下一轮 provider messages。
- Mutation：非 TTY 默认拒绝；显式 `--approval all` 才执行。
- Resume：同 workspace 恢复成功，跨 workspace 在 provider I/O 前失败。

实际 smoke run

```
{
  "protocol": "one-cli.events",
  "version": 1,
  "seq": 1,
  "type": "run.started",
  "model": "smoke-model"
}
{"protocol": "one-cli.events", "version": 1, "seq": 2, "type": "assistant.delta", "delta": "one-cli smoke run OK"}
{"protocol": "one-cli.events", "version": 1, "seq": 3, "type": "usage", "promptTokens": 8, "completionTokens": 5}
{"protocol": "one-cli.events", "version": 1, "seq": 4, "type": "assistant.completed", "interrupted": false}
{"protocol": "one-cli.events", "version": 1, "seq": 5, "type": "run.finished",
  "result": {"ok": true, "exitCode": 0, "reason": "completed", "rounds": 1}}
```

2026-08-07: 真实构建产物连接本地 OpenAI-compatible SSE 服务；进程退出码 0。当时环境未提供外部 OPENAI_API_KEY/OPENAI_MODEL，因此未调用真实远端模型。

发布验证

npm pack 仅包含 39 个文件，压缩包约 34.3 kB；GitHub 公开仓库已创建并推送： beforeload/one-cli。

10 后续路线图

优先级	动作	验收标准
P0	增加 OS sandbox backend；不可用时可禁用 shell	File/shell/child process/network 边界可验证
P0	补齐 SIGINT 进程级测试和 shell descendant teardown	首次取消、二次强退、退出码 130 稳定
P0	增加 provider 401/429/quota/Retry-After 矩阵	不可重试与可重试错误分类完整
P1	把 tools.ts 拆成 registry、runner、file、shell 模块	单文件职责和安全审查面进一步缩小
P1	加入 coverage、lint、format 与 GitHub Actions	核心安全路径有阈值，PR 自动门禁
P1	增加 session salvage/doctor，而非静默自愈	损坏状态可诊断、可导出、可人工选择
P2	在 execution kernel 稳定后评估 MCP	动态 tool schema 真正进入 agent loop
P2	按独立 Issue 增加 TUI 或 subagent	不破坏 CLI/headless transcript parity

建议的 Issue 模板

```
标题: Preserve partial assistant output on provider disconnect

前置条件: Provider 已产生至少一个 text_delta
行为: 连接异常终止
必须满足:
  1. 不发起第二次 provider request
  2. 持久化 assistant.state = interrupted
  3. JSONL 输出 assistant.completed(interrupted=true)
  4. run.finished.reason = provider_error
  5. 进程退出码 = 1
测试: unit scripted provider + integration broken SSE socket
```

11 结论

oh-my-cli 展示了一个 Coding Agent 如何通过数百个细粒度 Issue，把“会调用工具的聊天循环”演进为具备会话运营、自动化协议、安全控制面和自治研发流程的平台。它的先进之处是边界意识；它的技术债来自产品表面扩张快于核心整合。

对 Coding Agent 设计者

先定义 transcript、retry、approval、cancel、resume 和 exit semantics，再增加更多工具与 UI。Agent loop 很短，真正的工程复杂度集中在失败边界。

对 one-cli

当前 MVP 已证明最小 execution kernel 可在较小代码面内实现核心不变量。下一阶段应优先补 OS sandbox、provider 错误矩阵和 CI 质量门禁，而不是立即增加 MCP、TUI 或多 Agent。

最终判断

值得学习的不是 oh-my-cli 的全部功能，而是它如何把真实 dogfood 暴露的 partial、crash、cancel、quota、workspace mismatch 转化成可观察、可测试的运行协议。

A Issue 与代码证据索引

主题	代表 Issue	主要文件	测试形态
Partial stream / empty / quota	#243, #244, #247	src/agent.ts, src/provider.ts	Provider scripted unit + integration
Command policy / trust / path	#51, #87, #112	command-policy.ts, folder-trust.ts, workspace.ts	Policy matrix + symlink fixture
Read-only / cancellation	#234, #489, #550, #552	agent.ts, sigint-cancel.ts	Batch pairing + spawned CLI
Session continuity	#17, #243, #546, #554	session.ts, session-salvage.ts	Torn tail / corrupt / foreign workspace
Extensions	#118, #120, #135, #151	*-contract.ts, *-invocation.ts	Contract + dogfood capstone

主题	代表 Issue	主要文件	测试形态
Run caps	#515, #517	run-limits.ts, agent.ts	Injected clocks and counters
Headless strict semantics	#540, #552, #676–#692	headless-protocol.ts, index.ts	JSONL parse + exit code
Session operations	#592–#630	session-fork/search/archive/journal	Unit model + CLI integration
Automation hardening	#632–#708	journal/store/bundle modules	Filter ladder + strict mode

关键源码导航

仓库	文件	用途
oh-my-cli	src/agent.ts	主 agent loop、gate、取消、预算
oh-my-cli	src/provider.ts	Streaming、retry、quota 分类
oh-my-cli	src/tools.ts	内置工具、shell、原子写
oh-my-cli	src/workspace.ts	Canonical path 与 symlink 边界
oh-my-cli	src/session.ts	JSONL、恢复、checkpoint
oh-my-cli	src/headless-protocol.ts	NDJSON schema 与 terminal record
oh-my-cli	src/index.ts	CLI wiring 与大量诊断 surface
one-cli	src/agent.ts	精简的顺序状态机
one-cli	src/workspace.ts	统一文件 authority 与 stale check
one-cli	src/session.ts	严格 append-only journal

验证命令

```
cd /Users/daniel/workspace/one-cli
npm run typecheck
npm test
npm run test:integration
npm run check
npm pack --dry-run
npm audit --omit=dev --audit-level=high
```